

Lab 2

*Student: Preeti Pokharel Environment: Ubuntu Linux and WSL Date: September 24
2026*

TABLE OF CONTENT

Task 1 Hello World and Successful Exit Status

- Theoretical Concept
- Flow Diagram
- Source Code
- Compilation and Verification
- Summary

Task 2 Nonzero Exit Status

- Theoretical Concept
- Flow Diagram
- Source Code
- Compilation and Verification
- Summary

Task 3 Standard Input and Output

- Theoretical Concept
- Flow Diagram
- Source Code
- Compilation and Verification
- Summary

Task 4 Four Stages of GCC Compilation

- Theoretical Concept
- Flow Diagram
- Source Code
- Compilation and Verification
- Summary

Task 5 Returning Integer Value 50

- Theoretical Concept
- Flow Diagram
- Source Code
- Compilation and Verification
- Summary

Knowledge Check Questions

Conclusion

List of Figures

Figure 1 Source code for Hello World with return 0

Figure 2 Terminal verification for Task 1

Figure 3 Source code with return value 1

Figure 4 Terminal verification for Task 2

Figure 5 Source code for standard input and output

Figure 6 Terminal verification for Task 3

Figure 7 Source code used in the GCC compilation stages

Figure 8 Terminal verification of the four GCC stages

Figure 9 Source code returning integer value 50

Figure 10 Terminal verification for Task 5

Task 1 Hello World and Successful Exit Status

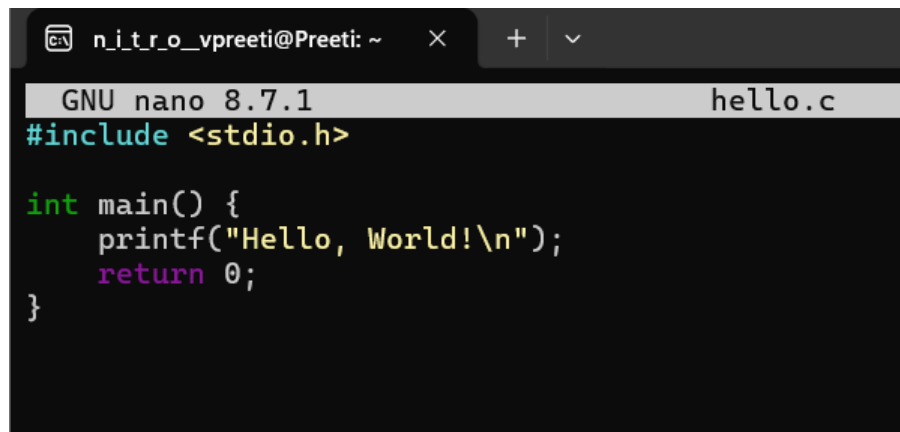
Theoretical Concept

A basic C program begins execution in the main function. The `stdio.h` header provides the declaration for `printf`. Returning 0 from main reports successful completion to the operating system, and the shell command `echo $?` displays that exit status.

Flow Diagram



Source Code



```

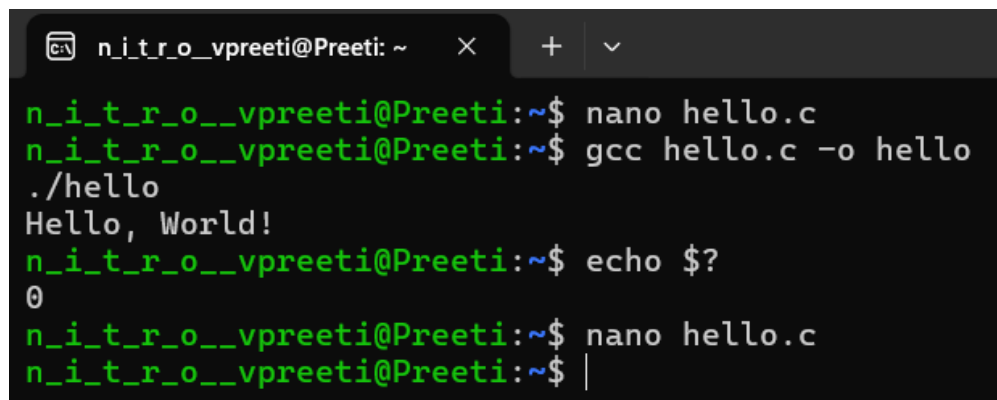
GNU nano 8.7.1 hello.c
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}

```

Figure 1: Source code for Hello World with return 0

Compilation and Verification



```

n_i_t_r_o__vpreeti@Preeti: ~$ nano hello.c
n_i_t_r_o__vpreeti@Preeti: ~$ gcc hello.c -o hello
./hello
Hello, World!
n_i_t_r_o__vpreeti@Preeti: ~$ echo $?
0
n_i_t_r_o__vpreeti@Preeti: ~$ nano hello.c
n_i_t_r_o__vpreeti@Preeti: ~$ |

```

Figure 2: Terminal verification for Task 1

Summary

The program compiled and printed Hello, World! The shell displayed exit status 0, confirming successful execution.

Task 2 Nonzero Exit Status

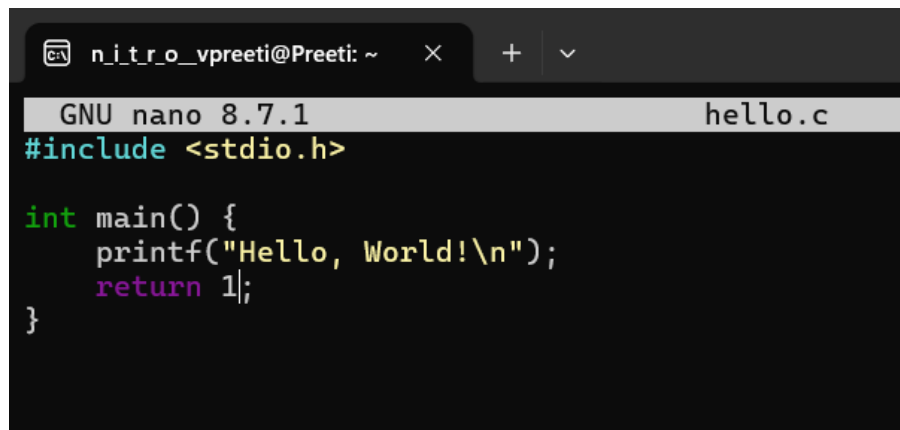
Theoretical Concept

The integer returned from main becomes the process exit status. By convention, 0 indicates success and a nonzero value indicates an error or another program-defined status. Changing the return value to 1 allows the shell to demonstrate this difference.

Flow Diagram



Source Code



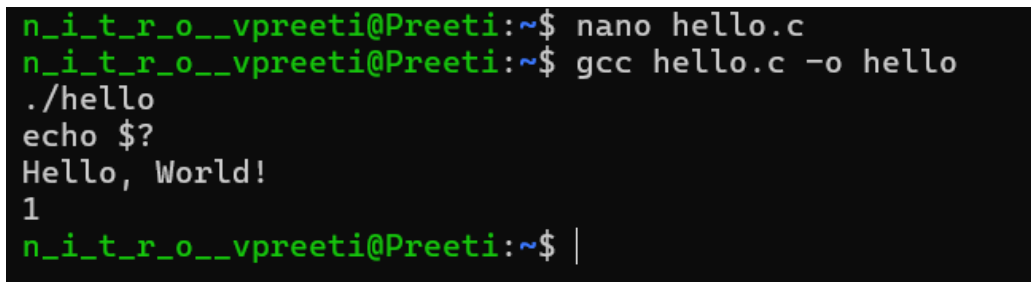
```

GNU nano 8.7.1 hello.c
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 1;
}
  
```

Figure 3: Source code with return value 1

Compilation and Verification



```

n_i_t_r_o__vpreeti@Preeti:~$ nano hello.c
n_i_t_r_o__vpreeti@Preeti:~$ gcc hello.c -o hello
./hello
Hello, World!
1
n_i_t_r_o__vpreeti@Preeti:~$ |
  
```

Figure 4: Terminal verification for Task 2

Summary

The program printed the expected message and echo \$? displayed 1, confirming that the nonzero value was passed to the operating system.

Task 3 Standard Input and Output

Theoretical Concept

The scanf function reads formatted data from standard input, normally the keyboard. The printf function writes formatted data to standard output, normally the terminal. The address operator & gives scanf the memory location where the entered integer must be stored.

Flow Diagram



Source Code

```

GNU nano 8.7.1 input.c *
#include <stdio.h>

int main() {
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    printf("You entered: %d\n", number);

    return 0;
}

```

Figure 5: Source code for standard input and output

Compilation and Verification

```

n_i_t_r_o__vpreeti@Preeti:~$ nano input.c
n_i_t_r_o__vpreeti@Preeti:~$ gcc input.c -o input
./input
Enter a number: 50
You entered: 50
n_i_t_r_o__vpreeti@Preeti:~$ |

```

Figure 6: Terminal verification for Task 3

Summary

The program accepted the integer 50 through standard input and correctly displayed the same value through standard output.

Task 4 Four Stages of GCC Compilation

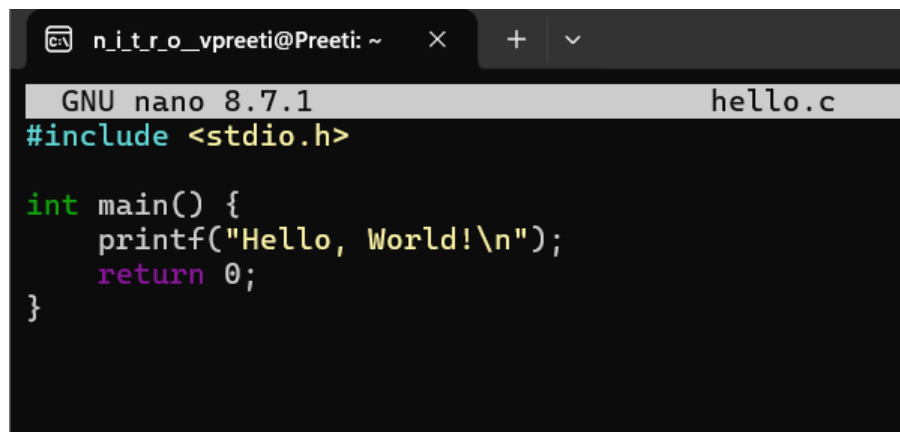
Theoretical Concept

GCC transforms C source code into an executable through four stages. Preprocessing expands directives and headers into a .i file. Compilation translates the result into assembly in a .s file. Assembling creates machine-code object data in a .o file. Linking combines the object file with required libraries to produce the executable.

Flow Diagram



Source Code



```

GNU nano 8.7.1 hello.c
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
  
```

Figure 7: Source code used in the GCC compilation stages

Compilation and Verification

```

n_i_t_r_o__vpreeti@Preeti:~$ gcc -E hello.c -o hello.i
n_i_t_r_o__vpreeti@Preeti:~$ gcc -S hello.i -o hello.s
n_i_t_r_o__vpreeti@Preeti:~$ gcc -c hello.s -o hello.o
n_i_t_r_o__vpreeti@Preeti:~$ gcc hello.o -o hello
n_i_t_r_o__vpreeti@Preeti:~$ ls -l hello.c hello.i hello.s hello.o hello
-rwxr-xr-x 1 n_i_t_r_o__vpreeti n_i_t_r_o__vpreeti 15952 Sep 24 16:28 hello
-rw-r--r-- 1 n_i_t_r_o__vpreeti n_i_t_r_o__vpreeti 80 Sep 24 16:20 hello.
c
-rw-r--r-- 1 n_i_t_r_o__vpreeti n_i_t_r_o__vpreeti 21435 Sep 24 16:27 hello.
i
-rw-r--r-- 1 n_i_t_r_o__vpreeti n_i_t_r_o__vpreeti 1496 Sep 24 16:28 hello.
o
-rw-r--r-- 1 n_i_t_r_o__vpreeti n_i_t_r_o__vpreeti 661 Sep 24 16:28 hello.
s
n_i_t_r_o__vpreeti@Preeti:~$ ./hello
Hello, World!
n_i_t_r_o__vpreeti@Preeti:~$
  
```

Figure 8: Terminal verification of the four GCC stages

Summary

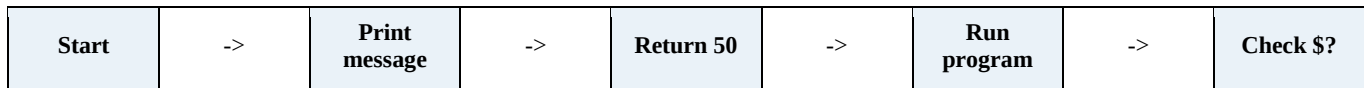
All four stages completed successfully. The listing confirmed that hello.i, hello.s, hello.o, and the final hello executable were created, and the executable printed Hello, World!

Task 5 Returning Integer Value 50

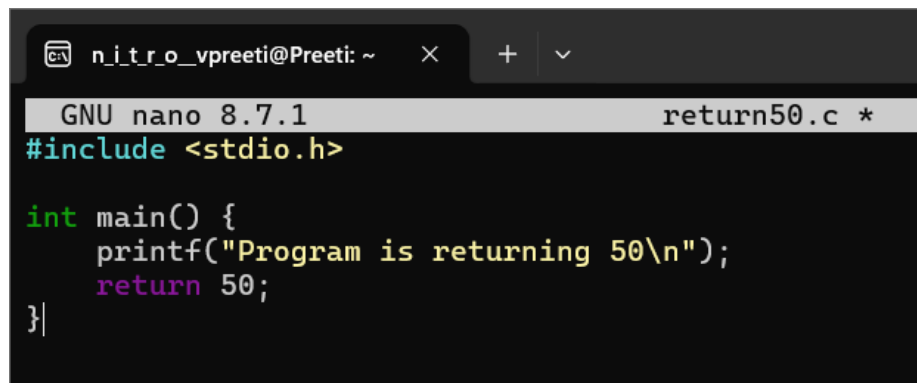
Theoretical Concept

A C program can return an integer status from main. The shell stores this status and makes it available through \$?. Returning 50 demonstrates that the operating system can receive a program-defined nonzero status value.

Flow Diagram



Source Code



```

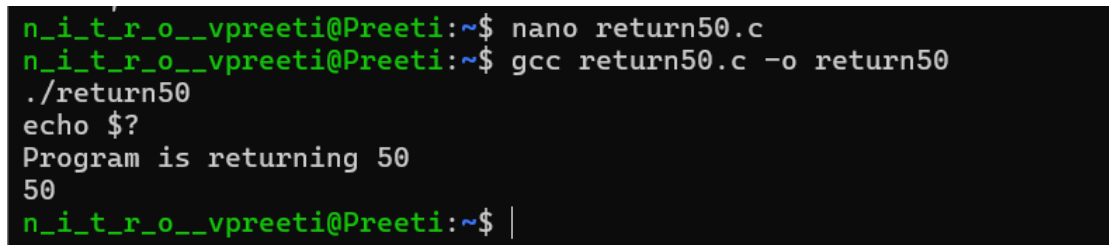
GNU nano 8.7.1 return50.c *
#include <stdio.h>

int main() {
    printf("Program is returning 50\n");
    return 50;
}

```

Figure 9: Source code returning integer value 50

Compilation and Verification



```

n_i_t_r_o__vpreeti@Preeti:~$ nano return50.c
n_i_t_r_o__vpreeti@Preeti:~$ gcc return50.c -o return50
./return50
echo $?
Program is returning 50
50
n_i_t_r_o__vpreeti@Preeti:~$ |

```

Figure 10: Terminal verification for Task 5

Summary

The program printed its message and returned 50. The shell command echo \$? displayed 50, verifying that the value was received as the exit status.

Knowledge Check Questions

Question 1 Purpose of the include directive

The `#include` directive inserts the contents of a header file before compilation. For example, `#include <stdio.h>` provides declarations for standard input and output functions such as `printf` and `scanf`.

Question 2 Four stages of compilation

Preprocessing expands headers and macros and produces a `.i` file. Compilation converts the preprocessed code into assembly and produces a `.s` file. Assembling converts assembly into object code and produces a `.o` file. Linking combines object code with required libraries and produces the final executable.

Question 3 Meaning of return values

A return value of 0 from `main` indicates successful completion. A nonzero return value normally represents an error, failure, or another special status defined by the program.

Question 4 Program that reads and displays an integer

The program in Task 3 declares an integer variable, asks the user for a value with `printf`, reads it using `scanf("%d", &number)`, and displays it using `printf("You entered: %d\n", number)`.

Question 5 GCC commands

Preprocess only: `gcc -E hello.c -o hello.i`

Generate assembly only: `gcc -S hello.c -o hello.s`

Create an object file without linking: `gcc -c hello.c -o hello.o`

Question 6 Compiled and interpreted languages

C is called a compiled language because a compiler translates the source code into machine-executable code before the program runs. An interpreted language is normally translated and executed by an interpreter while the program is running. Compiled programs generally execute faster because translation occurs before execution.

Conclusion

This lab demonstrated the structure of basic C programs, standard input and output, process exit values, and the GCC compilation process. The practical tasks confirmed that return 0 indicates successful execution, nonzero values can communicate other statuses, and GCC creates distinct preprocessing, assembly, object, and executable outputs. The completed programs compiled and ran correctly in Ubuntu Linux.

Submission Checklist

Requirement	Status
Hello World program and exit status 0	Completed
Program with exit status 1	Completed
Integer input and output program	Completed
Four GCC compilation stages	Completed
Program returning 50	Completed
Knowledge check questions	Completed
Screenshots included	Completed